

B/C/D 担当向け 実装開始マニュアル

授業マッチングアプリ 開発初心者向け

2026-06-08

目次

1. このマニュアルの目的	2
2. 全体の担当分担	3
3. 触ってよいファイル・注意するファイル	3
3.1 B 担当が主に触るファイル	3
3.2 C 担当が主に触るファイル	3
3.3 D 担当が主に触るファイル	3
3.4 勝手に変更しない方がよいファイル	4
4. 最初に各自がやること	4
4.1 リポジトリを取得する	4
4.2 パッケージをインストールする	4
4.3 環境変数ファイルを作成する	4
4.4 開発サーバを起動する	5
5. 作業開始前の確認	5
6. 作業ブランチの作り方	5
6.1 B 担当	6
6.2 C 担当	6
6.3 D 担当	6
7. 毎回の作業の流れ	6
8. B 担当: ログイン・プロフィールの進め方	7
8.1 B 担当の完成目標	7
8.2 実装順序	7
8.3 B 担当の動作確認	9
9. C 担当: 授業登録・時間割の進め方	9
9.1 C 担当の完成目標	9
9.2 実装順序	9
9.3 C 担当の動作確認	11

10. D 担当: マッチング・チャットの進め方	11
10.1 D 担当の完成目標	11
10.2 実装順序	12
10.3 D 担当の動作確認	13
11. Firebase を使うときの基本方針	14
11.1 Firebase 設定値について	14
11.2 Firestore の命名ルール	14
12. 動作確認の基本手順	15
12.1 開発サーバで確認	15
12.2 Console 確認	15
12.3 ビルド確認	15
13. Git 操作の基本	16
13.1 今の状態を見る	16
13.2 差分を見る	16
13.3 コミットする	16
13.4 push する	16
14. コミットメッセージの書き方	16
15. Pull Request 前チェックリスト	17
16. よくあるエラーと対処	17
16.1 npm run dev が失敗する	17
16.2 Failed to resolve import	18
16.3 Firebase: invalid-api-key	18
16.4 Cannot read properties of undefined	18
16.5 画面が真っ白になる	19
17. 実装時の基本ルール	19
17.1 画面ファイルに全部書きすぎない	19
17.2 まず小さく動かす	19
17.3 1 つの PR に詰め込みすぎない	19
18. B/C/D が A 担当に相談すべき場面	20
19. 最低限の完成ライン	20
B 担当	20
C 担当	20
D 担当	20
20. 最初に目指すデモの流れ	21

1. このマニュアルの目的

この文書は、A 担当が main に整備した初期環境を前提に、B/C/D 担当者が自分の担当機能を実装し始めるための手順をまとめたものです。

今回の main は、アプリ完成版ではありません。以下の目的で用意された開発用の土台です。

- npm install と npm run dev で起動できる
- Header から各仮ページへ移動できる
- 各担当が触るページファイルと service ファイルが分かれている
- Firebase を使うための src/firebase.js がある
- .env.example を見て各自が .env.local を作れる
- B/C/D が同じファイルを同時に触りにくい構成になっている

重要なのは、自分の担当ファイルを中心に編集し、共通ファイルは勝手に変更しないことです。

2. 全体の担当分担

担当	主な内容	主に触るファイル
A	GitHub 管理、画面遷移、共通レイアウト、README、初期環境	App.jsx, routes/, components/Layout.jsx, components/Header.jsx, README.md
B	ログイン、プロフィール	LoginPage.jsx, ProfilePage.jsx, authService.js, userService.js
C	授業登録、時間割、仮シラバス	TimetablePage.jsx, ClassSearchPage.jsx, classService.js, syllabus.js
D	マッチング、チャット	MatchingPage.jsx, ChatPage.jsx, matchingService.js, chatService.js

3. 触ってよいファイル・注意するファイル

3.1 B 担当が主に触るファイル

```
src/pages/LoginPage.jsx
src/pages/ProfilePage.jsx
src/services/authService.js
src/services/userService.js
```

3.2 C 担当が主に触るファイル

```
src/pages/TimetablePage.jsx
src/pages/ClassSearchPage.jsx
src/services/classService.js
src/data/syllabus.js
```

3.3 D 担当が主に触るファイル

```
src/pages/MatchingPage.jsx
src/pages/ChatPage.jsx
src/services/matchingService.js
src/services/chatService.js
```

3.4 勝手に変更しない方がよいファイル

以下は、複数人が触ると衝突しやすいファイルです。変更が必要な場合は、A 担当に相談してから編集してください。

```
src/App.jsx
src/main.jsx
src/routes/AppRoutes.jsx
src/components/Header.jsx
src/components/Layout.jsx
src/firebase.js
package.json
package-lock.json
README.md
.gitignore
```

特に `package.json` と `package-lock.json` は、ライブラリ追加時に変更されます。勝手に `npm install` ライブラリ名を行うと、他の人の作業と衝突しやすくなります。

4. 最初に各自がやること

4.1 リポジトリを取得する

まだ手元にリポジトリがない場合は、次を実行します。

```
git clone https://github.com/ユーザ名/リポジトリ名.git
cd リポジトリ名
```

すでに取得済みの場合は、最新版の `main` を取り込みます。

```
git switch main
git pull origin main
```

4.2 パッケージをインストールする

```
npm install
```

成功すると、`node_modules` が作成されます。`node_modules` は GitHub に上げません。

4.3 環境変数ファイルを作成する

プロジェクト直下に `.env.local` を作成します。

プロジェクト名/

```
├─ src/
├─ package.json
├─ .env.example
└─ .env.local
```

`.env.example` を参考にして、`.env.local` に Firebase 設定値を入れます。

```
VITE_FIREBASE_API_KEY=実際の値
VITE_FIREBASE_AUTH_DOMAIN=実際の値
VITE_FIREBASE_PROJECT_ID=実際の値
VITE_FIREBASE_STORAGE_BUCKET=実際の値
VITE_FIREBASE_MESSAGING_SENDER_ID=実際の値
VITE_FIREBASE_APP_ID=実際の値
```

`.env.local` は GitHub に上げません。`git status` に `.env.local` が出てきた場合は、すぐ A 担当に相談してください。

4.4 開発サーバを起動する

```
npm run dev
```

以下のような URL が表示されます。

```
http://localhost:5173/
```

ブラウザで開き、Header と仮ページが表示されれば OK です。

5. 作業開始前の確認

実装を始める前に、以下を確認してください。

```
npm install が成功する
npm run dev が成功する
http://localhost:5173/ が表示される
Header から自分の担当ページに移動できる
ブラウザの Console に赤いエラーが出ていない
```

ブラウザの Console は次の手順で確認します。

Chrome または Edge を開く

F12 を押す

Console タブを開く

赤色のエラーがないか確認する

警告は一旦無視してよいですが、赤色のエラーは修正対象です。

6. 作業ブランチの作り方

main に直接 push しないでください。各担当は、自分の作業ブランチを作ります。

6.1 B 担当

```
git switch main
git pull origin main
git switch -c feature/login-profile
```

6.2 C 担当

```
git switch main
git pull origin main
git switch -c feature/timetable
```

6.3 D 担当

```
git switch main
git pull origin main
git switch -c feature/matching-chat
```

すでにブランチを作成済みの場合は、新しく作り直す必要はありません。

7. 毎回の作業の流れ

開発初心者は、以下の流れを固定してください。

1. 自分のブランチにいることを確認
2. `npm run dev` で動作確認
3. 自分の担当ファイルを編集
4. ブラウザで確認
5. Console の赤いエラーを確認
6. `git status` で差分確認
7. `commit`

8. push

9. Pull Request 作成

コマンド例は以下です。

```
git branch
```

```
npm run dev
```

編集後に確認します。

```
git status
```

```
git add .
```

```
git commit -m "feat: ログイン画面を追加"
```

```
git push origin feature/login-profile
```

コミットメッセージの例は後述します。

8. B 担当: ログイン・プロフィールの進め方

B 担当は、最初から完璧なログイン機能を作ろうとせず、次の順に進めてください。

8.1 B 担当の完成目標

最低限の完成目標は以下です。

メールアドレスとパスワードで新規登録できる

ログインできる

ログアウトできる

ログイン中ユーザのプロフィールを登録できる

プロフィールを再表示できる

8.2 実装順序

■Step 1: LoginPage に入力欄を作る 対象ファイル:

```
src/pages/LoginPage.jsx
```

最初は Firebase と接続せず、画面だけ作ってください。

必要な UI:

メールアドレス入力欄

パスワード入力欄

新規登録ボタン

ログインボタン

ログアウトボタン

エラー表示欄

この時点の確認項目:

/login にアクセスできる
入力欄が表示される
ボタンが表示される
入力しても画面が壊れない
Console に赤いエラーが出ていない

■Step 2: authService に認証処理を書く 対象ファイル:

src/services/authService.js

このファイルには、Firebase Authentication を使う処理を書きます。

作る関数の目安:

```
signUp(email, password)
login(email, password)
logout()
observeAuthState(callback)
```

src/firebase.js から auth を import して使います。

```
import { auth } from "../firebase";
```

この時点の確認項目:

新規登録ボタンを押して Firebase Authentication にユーザが作成される
ログインボタンでログインできる
ログアウトボタンでログアウトできる
失敗時にエラー文が表示される

■Step 3: ProfilePage にプロフィール入力欄を作る 対象ファイル:

src/pages/ProfilePage.jsx

必要な UI:

ユーザ名
モチベーション
授業の受け方
保存ボタン
保存完了メッセージ

モチベーション例:

高め
普通
単位が取ればよい

授業の受け方例:

静かに受けない
話しながら受けない
授業後に相談したい
課題も一緒にやりたい

■Step 4: userService にプロフィール保存処理を書く 対象ファイル:

src/services/userService.js

作る関数の目安:

```
saveUserProfile(userId, profile)
getUserProfile(userId)
```

保存先のイメージ:

users/{userId}

データ例:

```
{
  name: "ユーザ名",
  motivation: "普通",
  studyStyle: "静かに受けない",
  createdAt: timestamp,
  updatedAt: timestamp
}
```

8.3 B 担当の動作確認

B 担当は、Pull Request を出す前に以下を確認してください。

/login が表示される

新規登録できる

ログインできる

ログアウトできる

/profile が表示される

プロフィールを保存できる

保存後に再読み込みしてもプロフィールが表示される

Console に赤いエラーがない

npm run build が成功する

9. C 担当: 授業登録・時間割の進め方

C 担当は、最初は Firestore を使わず、仮シラバスデータから画面を作ると進めやすいです。

9.1 C 担当の完成目標

最低限の完成目標は以下です。

授業コードを入力できる

仮シラバスデータから授業を検索できる

検索結果を表示できる

授業を時間割に追加できる

時間割画面に登録済み授業を表示できる

9.2 実装順序

■Step 1: 仮シラバスデータを確認する 対象ファイル:

src/data/syllabus.js

最初はこの仮データを使います。

```
export const syllabusData = [
  {
    code: "TEST101",
    name: "ソフトウェア開発論",
    day: "月",
    period: 3,
    teacher: "担当教員",
  },
];
```

本物のシラバス自動取得は後回しです。最初は仮データで動くことを優先してください。

■Step 2: ClassSearchPage で授業検索 UI を作る 対象ファイル:

src/pages/ClassSearchPage.jsx

必要な UI:

授業コード入力欄

検索ボタン

検索結果表示

時間割に追加ボタン

この時点では、検索結果を画面に表示するだけでもよいです。

確認項目:

/classes/search が表示される

TEST101 と入力して検索できる

ソフトウェア開発論が表示される

存在しないコードの場合に見つからない表示が出る

■Step 3: classService に検索処理を書く 対象ファイル:

src/services/classService.js

作る関数の目安:

```
findClassByCode(code)
getAllClasses()
```

最初は syllabusData から検索するだけで十分です。

■Step 4: 時間割への追加処理を作る 最初は useState だけで画面上に追加しても構いません。余裕があれば Firestore に保存します。

Firestore に保存する場合の関数の目安:

```
addClassToTimetable(userId, classData)
getMyTimetable(userId)
```

保存先の例:

users/{userId}/timetable/{classCode}

■Step 5: TimetablePage で表示する 対象ファイル:

src/pages/TimetablePage.jsx

必要な UI:

月～金

1～5 限

登録された授業名

授業コード

マッチング画面へ進むボタンまたはリンク

マッチング画面へのリンク例:

/matching/TEST101

9.3 C 担当の動作確認

C 担当は、Pull Request を出す前に以下を確認してください。

/classes/search が表示される

授業コードで検索できる

検索結果が表示される

時間割に追加できる

/timetable で授業が確認できる

/matching/TEST101 に移動できる

Console に赤いエラーがない
npm run build が成功する

10. D 担当: マッチング・チャットの進め方

D 担当が一番重い担当です。最初から完全なマッチングを作らず、段階的に進めてください。

10.1 D 担当の完成目標

最低限の完成目標は以下です。

- /matching/TEST101 で対象授業コードが表示される
- マッチングボタンを押せる
- 待機状態を Firestore に保存できる
- 同じ授業の待機者を取得できる
- 3 人集まったらグループを作成できる
- /chat/{groupId} に移動できる
- チャットメッセージを送信・表示できる

10.2 実装順序

■Step 1: MatchingPage の画面を作る 対象ファイル:

src/pages/MatchingPage.jsx

必要な UI:

- 対象授業コード
- マッチングボタン
- 現在の状態
- グループ作成後のチャット画面へのリンク

まずはボタンを押したら画面上の状態が変わるだけで構いません。

確認項目:

- /matching/TEST101 が表示される
- TEST101 が画面に表示される
- ボタンを押して状態が変わる
- Console に赤いエラーがない

■Step 2: matchingService に待機リクエスト処理を書く 対象ファイル:

src/services/matchingService.js

作る関数の目安:

```
createMatchRequest(userId, classCode, profile)
findWaitingRequests(classCode)
createGroup(classCode, members)
```

保存先のイメージ:

```
matchRequests/{requestId}
groups/{groupId}
```

matchRequests のデータ例:

```
{
  userId: "user001",
  classCode: "TEST101",
  motivation: "普通",
  studyStyle: "静かに受けたい",
  status: "waiting",
  createdAt: timestamp
}
```

groups のデータ例:

```
{
  classCode: "TEST101",
  members: ["user001", "user002", "user003"],
  createdAt: timestamp
}
```

最初は完全な同時処理対策をしなくてよいです。授業制作では、まず動くことを優先してください。

■Step 3: ChatPage の画面を作る 対象ファイル:

src/pages/ChatPage.jsx

必要な UI:

グループ ID

メッセージ一覧

メッセージ入力欄

送信ボタン

まずは仮メッセージを表示するだけでもよいです。

■Step 4: chatService に送受信処理を書く 対象ファイル:

src/services/chatService.js

作る関数の目安:

```
sendMessage(groupId, userId, text)
subscribeMessages(groupId, callback)
```

保存先のイメージ:

```
groups/{groupId}/messages/{messageId}
```

メッセージのデータ例:

```
{  
  userId: "user001",  
  text: "よろしくお願いします",  
  createdAt: timestamp  
}
```

Firestore のリアルタイム更新には onSnapshot を使うとよいです。

10.3 D 担当の動作確認

D 担当は、Pull Request を出す前に以下を確認してください。

- /matching/TEST101 が表示される
- マッチングボタンが押せる
- Firestore に matchRequests が作られる
- 条件を満たすと groups が作られる
- /chat/test-group または /chat/{groupId} が表示される
- メッセージを送信できる
- 送信したメッセージが表示される
- 再読み込み後もメッセージが表示される
- Console に赤いエラーがない
- npm run build が成功する

11. Firebase を使うときの基本方針

Firebase 関連の設定は src/firebase.js に集約されています。各担当は、必要に応じて以下のように import して使います。

```
import { auth, db } from "../firebase";
```

11.1 Firebase 設定値について

Firebase の設定値は .env.local に書きます。

```
VITE_FIREBASE_API_KEY=...  
VITE_FIREBASE_AUTH_DOMAIN=...  
VITE_FIREBASE_PROJECT_ID=...  
VITE_FIREBASE_STORAGE_BUCKET=...  
VITE_FIREBASE_MESSAGING_SENDER_ID=...  
VITE_FIREBASE_APP_ID=...
```

.env.local を変更した場合は、開発サーバを一度止めて再起動してください。

```
Ctrl + C  
npm run dev
```

11.2 Firestore の命名ルール

コレクション名は、次の方針で統一してください。

```
users  
matchRequests  
groups
```

チャットメッセージは、以下のようにグループのサブコレクションに置く方針です。

```
groups/{groupId}/messages/{messageId}
```

12. 動作確認の基本手順

各担当は、作業中に何度も以下を確認してください。

12.1 開発サーバで確認

```
npm run dev
```

ブラウザで確認します。

```
http://localhost:5173/
```

担当別の確認 URL:

```
B 担当: http://localhost:5173/login  
B 担当: http://localhost:5173/profile  
C 担当: http://localhost:5173/classes/search  
C 担当: http://localhost:5173/timetable  
D 担当: http://localhost:5173/matching/TEST101  
D 担当: http://localhost:5173/chat/test-group
```

12.2 Console 確認

ブラウザで F12 を押して、Console を確認します。

よく見るエラー:

```
Failed to resolve import  
Module not found  
Uncaught ReferenceError  
Firebase: Error  
Cannot read properties of undefined
```

赤いエラーがある場合は、PR を出す前に修正してください。

12.3 ビルド確認

Pull Request を出す前に必ず実行してください。

```
npm run build
```

成功すれば OK です。失敗した場合は、表示されたエラー文を読んで修正します。

13. Git 操作の基本

13.1 今の状態を見る

```
git status
```

見るべき点:

どのファイルを変更したか

`.env.local` が出ていないか

`node_modules` が出ていないか

担当外ファイルを変更していないか

13.2 差分を見る

```
git diff
```

13.3 コミットする

```
git add .
```

```
git commit -m "feat: 変更内容を短く書く"
```

13.4 push する

```
git push origin 自分のブランチ名
```

例:

```
git push origin feature/login-profile
```

14. コミットメッセージの書き方

種類	使う場面	例
feat	機能追加	feat: ログイン画面を追加

種類	使う場面	例
fix	バグ修正	fix: プロフィール保存時のエラーを修正
style	見た目調整	style: 時間割画面の表示を調整
docs	README 等	docs: 動作確認手順を追記
refactor	整理	refactor: 授業検索処理を service に分離
chore	設定・構成	chore: Firebase 設定を追加

初心者のうちは、完璧な名前にこだわりすぎなくてよいです。何をしたかが分かることを優先してください。

15. Pull Request 前チェックリスト

PR を出す前に、以下を確認してください。

- ☐ 自分の担当ブランチで作業している
- ☐ main に直接 push していない
- ☐ npm run dev が成功する
- ☐ npm run build が成功する
- ☐ 担当ページが表示される
- ☐ Console に赤いエラーがない
- ☐ .env.local を Git に追加していない
- ☐ node_modules を Git に追加していない
- ☐ 担当外ファイルを無断で変更していない
- ☐ README に影響する変更がある場合は A 担当に共有した

PR 本文には、最低限以下を書いてください。

変更内容

—

確認したこと

- ☐ npm run dev が成功する
- ☐ npm run build が成功する
- ☐ 担当ページが表示される
- ☐ Console に赤いエラーがない

注意点

—

16. よくあるエラーと対処

16.1 npm run dev が失敗する

まず、package.json があるフォルダで実行しているか確認してください。

`ls`

Windows PowerShell の場合:

`dir`

package.json が見えない場所で npm run dev を実行しても失敗します。

16.2 Failed to resolve import

例:

```
Failed to resolve import "../services/authService"
```

原因の例:

ファイル名が違う

フォルダの位置が違う

import のパスが違う

拡張子や大文字小文字が違う

確認すること:

src/services/authService.js が存在するか

import 文の ../ の数が合っているか

ファイル名の大文字小文字が一致しているか

16.3 Firebase: invalid-api-key

原因の例:

.env.local がない

.env.local の値が空

変数名が違う

npm run dev を再起動していない

対処:

.env.local を確認する

VITE_ から始まる変数名になっているか確認する

Ctrl + C で開発サーバを止める

npm run dev で再起動する

16.4 Cannot read properties of undefined

例:

Cannot read properties of undefined (reading 'name')

原因の例:

まだ取得できていないデータを画面で表示しようとしている

Firestore から null が返っている

state の初期値が想定と違う

対処例:

```
{profile && <p>{profile.name}</p>}
```

または、初期値を入れておきます。

```
const [profile, setProfile] = useState(null);
```

16.5 画面が真っ白になる

確認順:

Console に赤いエラーがないか

AppRoutes.jsx のルートが間違っていないか

ページコンポーネントが export default されているか

import のパスが間違っていないか

17. 実装時の基本ルール

17.1 画面ファイルに全部書きすぎない

悪い例:

LoginPage.jsx の中に Firebase 処理、画面、バリデーションを全部書く

よい例:

LoginPage.jsx は画面とボタン処理を担当

Firebase 処理は authService.js に分ける

17.2 まず小さく動かす

最初から完成形を作ろうとしないでください。

よい進め方:

画面だけ作る

ボタンを押したら `console.log` する

`service` 関数を呼ぶ

Firestore に保存する

保存結果を画面に表示する

17.3 1 つの PR に詰め込みすぎない

悪い PR:

ログイン、プロフィール、時間割、デザイン変更を全部まとめる

よい PR:

ログイン画面だけ

プロフィール保存だけ

授業検索だけ

チャット送信だけ

18. B/C/D が A 担当に相談すべき場面

以下の場合、自分だけで進めず A 担当に相談してください。

ルートを追加したい

Header にリンクを追加したい

新しいライブラリを入れたい

`package.json` を変更する必要がある

`firebase.js` を変更したい

他担当のファイルに触る必要がある

main との merge でコンフリクトした

`.env.local` が Git に入りそうになった

19. 最低限の完成ライン

B 担当

ログイン画面が動く

新規登録・ログイン・ログアウトができる

プロフィールを保存・取得できる

C 担当

授業コード検索ができる

授業を時間割に追加できる

時間割画面に授業が表示される

D 担当

マッチングボタンを押せる

待機情報を保存できる

グループを作成できる

チャット画面でメッセージを送受信できる

全員共通:

npm run dev が成功する

npm run build が成功する

Console に赤いエラーがない

自分の担当外ファイルを無断で壊していない

20. 最初に目指すデモの流れ

最終発表・授業内デモでは、まず以下の流れが見せられれば十分です。

1. ユーザがログインする
2. プロフィールを登録する
3. 授業コード TEST101 を検索する
4. 時間割に追加する
5. TEST101 のマッチング画面を開く
6. マッチングボタンを押す
7. グループが作られる
8. チャット画面に移動する
9. メッセージを送信する

この流れを実現するために、各担当は自分の範囲を小さく完成させてください。